

Autonomous Collision-Avoidance Subsystem for AGVs

Design, Kinematic Simulation, and POSIX Linux Safety Controller

#1



B.Tech Final Year Capstone Project

A real-time robotics safety controller in modern C++17 featuring dynamic stopping distance physics, Artificial Potential Field (APF) autonomous swerving, and POSIX Linux device driver emulation.

C++17 Engine

APF Swerve Steering

/dev/agv_radar Driver

100% Test Verified

Industry Problem & Motivation

Why Fixed-Distance Sensors Fail on Mobile Robots

#2

✗ Traditional Fixed Cutoff Sensors

- Fixed 1-meter cutoffs cause collisions at high closing speeds.
- Triggers false emergency stops for receding objects or side shelves.
- Causes severe throughput loss and jerky stop-and-go motion.
- Cannot perform autonomous local bypass maneuvers.

✓ Our Dynamic Multi-Tier Solution

- Stopping distance scales dynamically with instant vehicle velocity.
- Time-To-Collision (TTC) prevents false alarms on receding targets.
- Swept-volume geometry checks full obstacle radius to prevent clipping.
- Artificial Potential Fields (APF) steer smoothly around obstacles.

Project Aim & Objectives

Key Technical Milestones Achieved

#3

1. Kinematic Modeling

Model differential-drive unicycle kinematics with acceleration and deceleration limits.

3. 4-State Safety FSM

Implement NORMAL, WARNING, SLOWDOWN, EMERGENCY_STOP with recovery debouncing.

2. Dynamic Risk Engine

Implement physical stopping distance equations and Time-to-Collision (TTC).

4. Embedded Device Driver

Emulate a POSIX Linux character device driver (`/dev/agv_radar`) with thread-safe ring buffering.

System Architecture & Block Diagram

#4

Unidirectional Decoupled Pipeline



World

2D Kinematics



LiDAR

64-Ray Scan



Device Driver

/dev/agv_radar



Risk Engine

TTC & Distance



Safety FSM

4-State Machine



Controller

APF + Decel

Hardware Decoupling: The safety controller reads exclusively from the Linux character device buffer. Replacing the virtual sensor with real physical radar requires zero code changes to the safety algorithm.

Dynamic Stopping Distance Formulation

Kinematic Derivation of Safe Braking Boundaries

$$d_{\text{stop}}(v) = v \cdot t_r + (v^2 / 2a) + d_s$$

Reaction Distance (0.2s latency) + Braking Distance ($a = 1.2 \text{ m/s}^2$) + Safety Buffer (0.3m)

1. Emergency Zone ($\leq d_{\text{stop}}$)

Full deceleration (1.2 m/s^2). Speed target = 0 m/s.

2. Slowdown Zone ($\leq d_{\text{stop}} + 1.2\text{m}$)

Reduce speed to 45% (0.54 m/s) for safe approach.

3. Warning Zone ($\leq d_{\text{stop}} + 3.4\text{m}$)

Alert logged to journal, cruise speed maintained.

Time-To-Collision (TTC) Calculation

Handling Moving Obstacles and Eliminating False Alarms

#6

TTC Equation

```
v_closing = -( x.relVx + y.relVy ) / dist )  
TTC = surfaceDist / v_closing
```

Projects relative motion onto the line-of-sight axis to determine exact contact time.

Safety Thresholds

- **TTC ≤ 1.0s**: Immediate EMERGENCY STOP.
- **TTC ≤ 2.5s**: Proactive SLOWDOWN transition.
- **v_closing ≤ 0**: Receding object → TTC = ∞ (0 false stops).

Swept-Volume Corridor Detection

Eliminating Overlap for Off-Track Protrusions

1. Local Frame Coordinate Transform

```
localX = dx*cos(-θ) - dy*sin(-θ)
localY = dx*sin(-θ) + dy*cos(-θ)
```

Transforms global world coordinates into AGV forward (+X) and lateral (+Y) frame.

2. True Swept-Volume Boundary

```
lateralOverlap = |localY| - obs_radius
inCorridor = (lateralOverlap ≤ 0.55m)
```

Subtracts obstacle radius to detect any obstacle edge intruding into the travel lane.

Artificial Potential Field (APF) Navigation

Autonomous Real-Time Swerving & Gridlock Safety

APF Force Equations

Attraction (Lane Center): $F_{att_y} = -agv.y \cdot 1.8$

Forward Cruise: $F_{att_x} = 3.0$

Repulsion (Obstacles): $F_{rep} = 20.0 / (dist - r)^2$

Gridlock Wall Handling

When multiple obstacles form an impassable barricade across all lanes, repulsive forces cancel out.

Solution: 1D longitudinal interval merging detects barricades and commands straight emergency braking.

Linux Character Device Interface

POSIX System Programming Concepts (/dev/agv_radar)

Binary Hardware Packet Structure

| Magic: 0x52414452 | Timestamp (64-bit) | Distance (32-bit) | Azimuth (16-bit) | Speed (16-bit) | XOR Checksum |

ARM Safe

Uses `std::memcpy` to avoid unaligned pointer SIGBUS faults on ARM CPUs.

Mutex Protected

Thread-safe circular ring buffer prevents race conditions between sensor and FSM.

POSIX APIs

Implements `dev_open()`, `dev_close()`, `dev_read()`, `dev_write()`.

Simulation Test Cases & Verification

6 Core Scenarios Verified in Interactive Simulator

Scenario	Setup	Observed Behavior	Status
1. Free Cruise	Empty Aisle	Cruise at nominal 1.2 m/s; 0 false stops	PASS
2. Center Obstacle	Obstacle at (5.5m, 0.0m)	Smooth APF swerve bypass & lane return	PASS
3. Off-Track Protrusion	Obstacle at (4.8m, 0.8m)	Swept-volume detects edge; swerves safely	PASS
4. Dynamic Crossing	Moving pedestrian at 0.4 m/s	TTC triggers slowdown; clean passage	PASS
5. Slalom S-Curve	3 staggered obstacles	Continuous smooth weaving without stopping	PASS
6. Full Gridlock	3-obstacle barricade wall	Gridlock detected; straight stop with +0.37m margin	PASS

Engineering Challenges & Root Causes

#11

Real Issues Solved During Implementation

1. Threshold Hunting Oscillation:

Braking reduced speed, which shrank stopping distance, causing rapid start-stop cycling. *Fix:* Evaluated thresholds at `max(v_actual, v_nominal)`.

2. Slalom False Gridlock Bug:

Obstacles spaced 3.5m apart were merged into a single wall. *Fix:* Grouped obstacles by longitudinal X slice ($|x1 - x2| \leq 1.5\text{m}$).

3. Recovery Hysteresis Deadband:

Sensor noise caused chattering after clearing E-stop. *Fix:* Added a `0.8s` confirmation timer and `+0.5m` buffer.

Conclusion & Future Roadmap

Project Summary and Hardware Path

#12

✓ Project Achievements

Full C++17 modular safety framework, POSIX driver layer, and interactive APF web simulator with 100% test pass rate.

🚀 Future Scope

ROS 2 Lifecycle node integration, Gazebo 3D simulation, and hardware deployment on an NVIDIA Jetson with physical automotive radar.

Thank you! Ready for Questions.